



AarogyaHub

Telemedicine & Medical Records Platform

System Architecture & Design Document

Roles | Architecture | Project Structure | ER Diagram | Data Flows | APIs | DICOM | Deployment

Version 1.0 (MVP) | 30 September 2026

Product name candidates

Name	What it suggests	Proposed use in this project
AarogyaHub (recommended)	Aarogya = health (Hindi/Sanskrit); a hub for patients, doctors and records	Master platform brand used throughout this document
MedVault	Secure, private storage	Name for the medical records + DICOM vault module
MedGo	Speed, instant access	Name for a future mobile app / instant-consult feature
MediCart	Shopping / pharmacy	Reserve for a future e-pharmacy module (not MVP)

Using one umbrella brand with named modules keeps the domain, logo and app-store listing simple while still using all four ideas. Check trademark and domain availability before finalising, since several of these names are generic and may already be registered.

Document contents

1 Product scope and prerequisites	8 API reference
2 Technology stack	9 DICOM medical imaging
3 User roles and permissions (RBAC)	10 LLM chatbot (RAG)
4 High-level architecture diagram	11 Security and compliance
5 Project structure (codebase layout)	12 Deployment on Vercel
6 Database design and ER diagram	13 Testing with Postman
7 Data flows for key features	14 Roadmap and milestones

1. Product scope and prerequisites

AarogyaHub is a telemedicine platform where patients discover verified doctors, book time-slot appointments, consult over video, and share medical reports - including DICOM medical images such as X-ray, CT and MRI. An LLM chatbot answers FAQs and guides users through the app. Admins verify doctors before they can be booked.

1.1 MVP feature set

Module	What is included in the MVP
Authentication	Email/phone + password sign-up, JWT access + refresh tokens, email OTP verification, password reset, role selection (patient / doctor)
Doctor onboarding	Profile, licence and degree upload, admin verification (pending / verified / rejected), re-submission after rejection
Availability & slots	Weekly availability per day; doctor chooses slot duration (5/10/15/20/30 min) per availability block; date-specific leave
Doctor discovery	Search and filter by specialization, language, fee and next available slot; only verified, active doctors are listed
Appointments	Book, reschedule, cancel; double-booking prevented at database level; reminders by email/SMS
Video consultation	Managed WebRTC room per appointment, waiting room, join window (e.g. 10 min before start)
Reports (MedVault)	Upload PDF / images / DICOM (.dcm) ; share per appointment; doctor views only reports shared with them
DICOM viewer	In-browser viewer with window/level, zoom, pan, series scroll; metadata extracted on the server
Consultation notes	Structured text notes (SOAP) written by the doctor, visible to the patient
LLM chatbot	RAG over admin-curated FAQ and health content; non-clinical guardrails; emergency detection
Admin console	Doctor verification queue, user blocking, appointment override, knowledge-base editor, audit log viewer, basic analytics

1.2 Prerequisites

Area	Requirement
Developer machine	Node.js 20 LTS + npm/pnpm, Python 3.12, Git, VS Code (or similar), Postman desktop, Docker (optional, for local MongoDB/Redis)
Accounts	GitHub, Vercel, MongoDB Atlas (free M0 cluster for testing), object storage (AWS S3 or Cloudflare R2), video provider (100ms / Daily / Agora), LLM API key, SendGrid or SES, MSG91/Twilio
Medical imaging	Sample anonymised DICOM files for testing (public datasets such as The Cancer Imaging Archive); never use real patient DICOM in development
Knowledge	REST and JWT basics, React hooks, async Python, MongoDB document modelling, WebRTC concepts (handled mostly by the provider SDK)
Legal / domain	Doctor licence verification against the relevant medical council register; awareness of India's Telemedicine Practice Guidelines (2020) and the Digital Personal Data Protection Act, 2023

2. Technology stack

Layer	Choice	Why
Frontend	React 18 + Vite, TypeScript, React Router, TanStack Query, Zustand, Tailwind CSS, React Hook Form + Zod	Fast builds, typed forms, server-state caching; deploys as static files on Vercel
DICOM viewing	Cornerstone3D (+ dicom-image-loader); OHIF viewer as a later upgrade	Open-source, browser-native medical image rendering
Video	Provider React SDK (100ms / Daily / Agora)	Avoids running STUN/TURN and signalling servers, which Vercel cannot host
Backend	FastAPI, Pydantic v2, Uvicorn (local), Beanie ODM on Motor	Async, auto-generated OpenAPI docs, clean validation
Auth	JWT (python-jose / PyJWT), passlib[bcrypt] or argon2	Stateless auth that suits serverless functions
Database	MongoDB Atlas (testing and production tiers)	Flexible documents for profiles and DICOM metadata; Atlas Vector Search for RAG
Files	S3 / Cloudflare R2 with pre-signed URLs	Uploads go straight from the browser to storage, bypassing function size limits
Cache / limits	Upstash Redis (HTTP-based)	Works from serverless; rate limits, OTPs, slot locks
Imaging libs	pydicom, Pillow, numpy	Parse DICOM tags, anonymise, generate thumbnails
AI	LLM API + embeddings API	Chatbot answers grounded in curated content
Testing	Postman collections + Newman in CI, pytest + httpx, Vitest + React Testing Library	API contract tests plus unit tests
Deployment	Vercel (frontend + Python functions), GitHub Actions	Preview deployment per pull request

3. User roles and permissions

Four roles drive role-based access control (RBAC) across the UI, the API and the data. Every API route declares the roles it accepts, and every query is additionally scoped by **ownership** (a patient only sees their own records, a doctor only sees their own appointments).

Visitor - Unauthenticated user browsing the site.

- + **Can:** View landing page, features, FAQs and public health articles; browse verified doctors (no contact details); use chatbot for general FAQs; register as patient or doctor.
- **Cannot:** Book appointments, join calls, upload or view reports, access dashboards.

Patient - Registered user with a patient profile.

- + **Can:** Complete profile and basic medical history; search/filter doctors; book, reschedule and cancel appointments; upload reports and DICOM studies; share reports with a doctor per appointment; join video call; view own history and consultation notes; use chatbot with light personalisation.
- **Cannot:** See other patients' data, edit doctor profiles, access admin features.

Doctor - Medical professional. Inactive until an admin verifies the account.

- + **Can: Unverified:** edit profile, upload licence documents, see verification status.
Verified: set weekly availability with own slot duration; view own schedule; join consultations; view reports shared for their appointments; write consultation notes.
- **Cannot:** Create availability or receive bookings while unverified; see other doctors' schedules or patients; change system settings.

Admin - Platform operator / internal team.

- + **Can:** Verify or reject doctors; block/unblock users; override or cancel appointments in exceptional cases; manage chatbot knowledge base; view audit logs and analytics; configure fees and feature flags.
- **Cannot:** Perform clinical actions; open medical files without an audit record (every access is logged with a reason).

3.1 Permission matrix

Action	Visitor	Patient	Doctor (unverified)	Doctor (verified)	Admin
Browse verified doctors	Yes	Yes	Yes	Yes	Yes
Use chatbot	FAQ only	Yes	Yes	Yes	Yes
Edit own profile	-	Own	Own	Own	Own
Upload verification documents	-	-	Yes	Yes	-
Verify / reject doctors	-	-	-	-	Yes
Create availability (slot duration)	-	-	-	Own	-
Book / cancel appointment	-	Own	-	Cancel own	Override
Join video consultation	-	Own	-	Own	-
Upload reports / DICOM	-	Own	-	-	-
View reports	-	Own	-	Shared with them	Logged access
Write consultation notes	-	Read own	-	Own	-
Manage knowledge base	-	-	-	-	Yes
View audit logs / analytics	-	-	-	-	Yes
Block users	-	-	-	-	Yes

3.2 How RBAC is enforced

```
# app/core/deps.py
def require_roles(*roles: Role):
    async def checker(user: User = Depends(get_current_user)) -> User:
        if user.role not in roles or not user.is_active:
            raise HTTPException(403, "Not allowed")
        return user
    return checker

async def require_verified_doctor(user = Depends(require_roles(Role.DOCTOR))):
    doctor = await Doctor.find_one(Doctor.user_id == user.id)
    if not doctor or doctor.verification_status != "verified":
        raise HTTPException(403, "Doctor account not verified yet")
    return doctor

# usage
@router.post("/doctors/me/availability")
async def create_availability(body: AvailabilityIn, doctor = Depends(require_verified_doctor)): ...
```

The frontend mirrors these rules with a `<RequireRole>` route guard, but it only hides UI; the backend check is the real security boundary.

4. High-level architecture

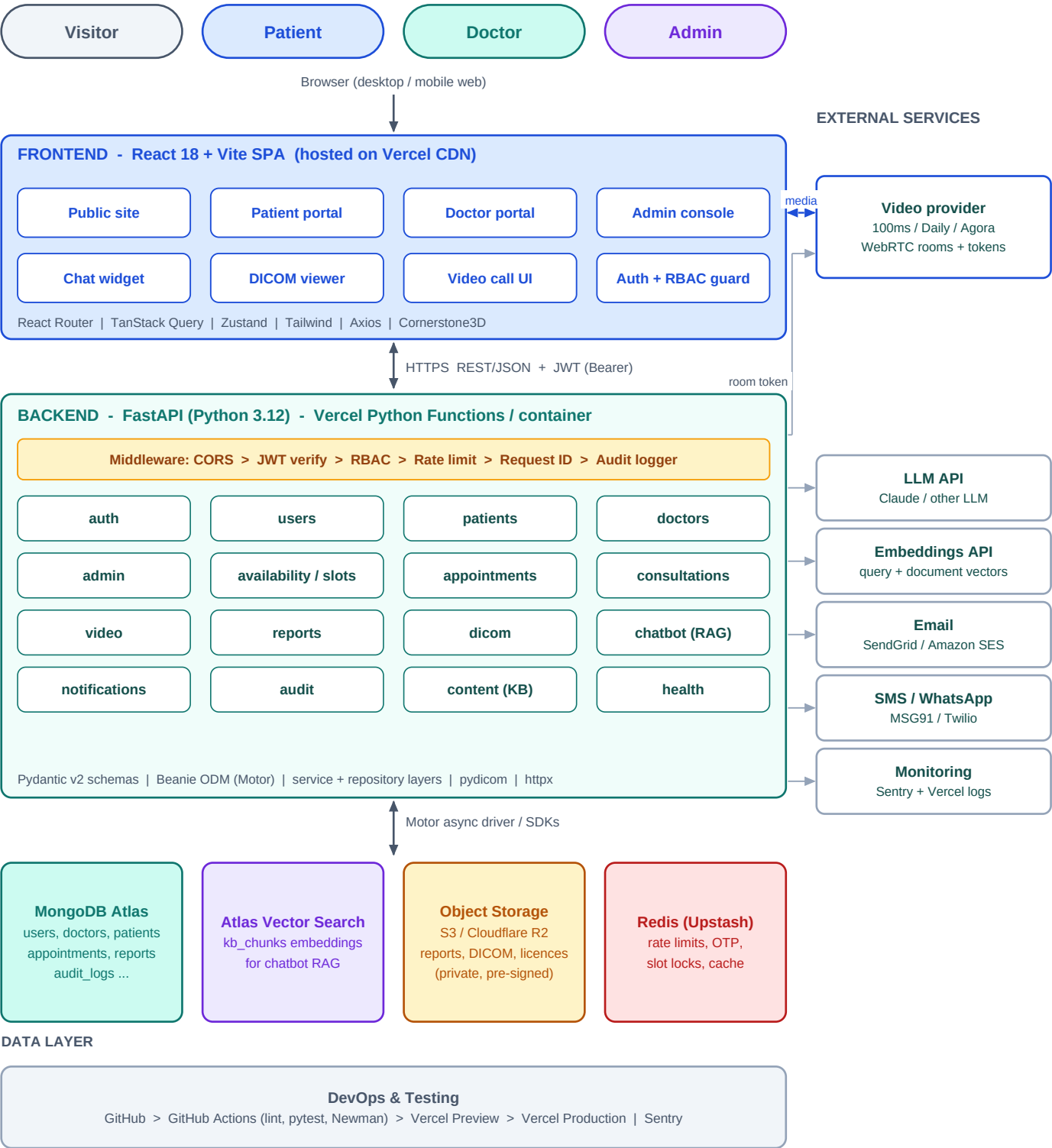


Figure 1 - AarogyaHub system architecture (MVP)

4.1 Component responsibilities

Component	Responsibility
React SPA	One app with route groups: / public, /patient, /doctor, /admin. Lazy-loaded per role so patients never download admin code.
FastAPI backend	Modular monolith: each domain module has router, schemas, service and model. Easy to split into microservices later.
Middleware	Validates JWT, attaches user, enforces rate limits (Redis), adds request ID, writes audit entries for sensitive routes.
MongoDB Atlas	System of record for users, profiles, availability, appointments, report metadata, chat, audit logs.
Object storage	Private bucket for reports, DICOM files and licence documents. Accessed only through short-lived pre-signed URLs.
Atlas Vector Search	Stores embeddings of knowledge-base chunks; top-k similarity search for the chatbot.
Redis	Rate limiting, OTP codes, short slot locks during checkout, cached doctor listings.
Video provider	Hosts WebRTC rooms, TURN relays and recordings (recording off by default). Backend only issues room tokens.
Notifications	Email and SMS for verification results, booking confirmations and reminders.

Design principle: the browser talks to the backend for decisions (who may do what) and talks directly to storage and the video provider for heavy data (files and media). This keeps serverless functions small, fast and cheap.

5. Project structure

```

aarogyahub/
├── frontend/                                # React + Vite + TypeScript
│   ├── public/
│   ├── src/
│   │   ├── app/
│   │   │   ├── router.tsx                # route tree, lazy role bundles
│   │   │   ├── providers.tsx             # QueryClient, theme, auth
│   │   │   └── RequireRole.tsx           # route guard
│   │   ├── api/
│   │   │   ├── client.ts                 # axios client + typed endpoints
│   │   │   ├── auth.ts doctors.ts appointments.ts
│   │   │   ├── reports.ts dicom.ts chatbot.ts admin.ts
│   │   ├── features/
│   │   │   ├── public/                  # landing, doctor list, FAQs, blog
│   │   │   ├── auth/                   # login, register, OTP, reset
│   │   │   ├── patient/                 # dashboard, booking, history, profile
│   │   │   ├── doctor/                 # onboarding, verification status,
│   │   │   │   └── availability editor, schedule, notes
│   │   │   ├── admin/                  # verification queue, users, KB, audit
│   │   │   ├── consultation/            # waiting room, VideoRoom.tsx
│   │   │   ├── reports/                # upload, list, share dialog
│   │   │   ├── dicom/                  # DicomViewer.tsx (Cornerstone3D)
│   │   │   └── chatbot/                 # ChatWidget.tsx
│   │   ├── components/ui/              # Button, Modal, Table, SlotPicker ...
│   │   ├── hooks/                      # useAuth, useUpload, useSlots
│   │   ├── store/                      # zustand: auth, ui
│   │   ├── lib/                        # dates (UTC <-> IST), validators
│   │   └── main.tsx
│   ├── .env.example
│   ├── vite.config.ts tailwind.config.ts package.json
│   └── vercel.json                      # SPA rewrites
├── backend/                               # FastAPI
│   ├── api/
│   │   └── index.py                     # Vercel entry: from app.main import app
│   ├── app/
│   │   └── main.py                      # create_app(), routers, middleware
│   ├── core/
│   │   ├── config.py                   # pydantic-settings (env vars)
│   │   ├── security.py                 # hashing, JWT encode/decode
│   │   ├── deps.py                     # get_current_user, require_roles
│   │   ├── db.py                       # Motor client + Beanie init
│   │   ├── storage.py                  # S3/R2 pre-signed URL helpers
│   │   └── redis.py ratelimit.py audit.py errors.py
│   ├── modules/
│   │   ├── auth/                       router.py schemas.py service.py
│   │   ├── users/                     router.py schemas.py service.py models.py
│   │   ├── patients/                  ...
│   │   ├── doctors/                   ... verification.py
│   │   ├── availability/               ... slots.py          # slot generation
│   │   ├── appointments/              ...
│   │   ├── consultations/             ... notes.py
│   │   ├── video/                     provider.py          # room + token
│   │   ├── reports/                   ...
│   │   ├── dicom/                     parser.py anonymize.py thumbnails.py
│   │   ├── chatbot/                   rag.py guardrails.py prompts.py
│   │   ├── kb/                        ingest.py chunker.py embeddings.py
│   │   ├── notifications/             email.py sms.py templates/
│   │   └── admin/                     router.py analytics.py
│   ├── utils/                          # time, pagination, ids
│   ├── scripts/
│   │   ├── seed.py                    # admin, demo doctors, patients
│   │   └── create_indexes.py
│   ├── tests/                          # pytest: unit + API tests
│   ├── requirements.txt pyproject.toml
│   ├── .env.example
│   └── vercel.json                     # python runtime, routes -> api/index.py
├── postman/
│   ├── AarogyaHub.postman_collection.json
│   ├── local.postman_environment.json
│   └── staging.postman_environment.json
├── docs/                                # this document, ADRs, API notes
├── .github/workflows/ci.yml             # lint, pytest, newman, build
├── docker-compose.yml                   # local MongoDB + Redis
└── README.md

```

Why a monorepo: a single Git repository keeps frontend, backend, Postman collections and docs versioned together. Vercel builds both apps from the same repo.

5.1 Backend module pattern

Every module follows the same four-file pattern so any developer can find code quickly:

File	Contains	Rule
models.py	Beanie Document classes (MongoDB collections) and indexes	No business logic
schemas.py	Pydantic request/response models (e.g. AppointmentCreate, AppointmentOut)	Never return raw DB documents
service.py	Business rules: slot validation, verification checks, notifications	Unit-tested, no FastAPI imports
router.py	HTTP routes, dependency injection, status codes	Thin - calls the service

6. Database design (MongoDB)

MongoDB is document-based, so 'relations' are references by ObjectId. The ER diagram below shows collections as entities and references as relationships. Collections that belong to a single user (chat sessions, audit logs, notifications) also carry a user_id reference to users; those lines are left out to keep the diagram readable.



Figure 2 - Entity-relationship diagram (collections, key fields and cardinality)

6.1 Collections summary

Collection	Purpose	Key indexes
users	Login identity and role	email unique; phone unique (sparse)
patients / doctors	Role-specific profiles (1:1 with users)	user_id unique; doctors: {verification_status, specialization}, text index on name/specialization
doctor_documents	Licence, degree, ID proofs for verification	doctor_id
doctor_availability	Weekly blocks with slot_duration_min	{doctor_id, day_of_week}
doctor_leaves	Date-specific unavailability (not drawn)	{doctor_id, date}
appointments	Bookings with status lifecycle	unique partial {doctor_id, start_time} where status in [scheduled, in_progress]; {patient_id, start_time}
consultations / consultation_notes	Video session state and doctor notes	appointment_id unique; consultation_id
reports / dicom_studies	File metadata and DICOM tags	{patient_id, created_at}; study_instance_uid
chat_sessions / chat_messages	Chatbot history (retention-limited)	session_id; TTL index on created_at (e.g. 90 days)
kb_articles / kb_chunks	Curated content and embeddings	slug unique; Atlas vector index on embedding
notifications	In-app notification feed (not drawn)	{user_id, read, created_at}
audit_logs	Append-only record of sensitive actions	{entity_type, entity_id}; {actor_user_id, created_at}

6.2 Enumerations

Field	Values
users.role	patient, doctor, admin (visitors are not stored)
doctors.verification_status	pending -> verified rejected; rejected -> pending (after re-upload); verified -> suspended (admin)
doctor_availability.slot_duration_min	5, 10, 15, 20, 30
appointments.status	scheduled -> in_progress -> completed; scheduled -> cancelled no_show
consultations.status	scheduled, waiting, live, ended
reports.type	pdf, image, dicom

6.3 Sample documents

```
// doctors
{ "_id": ObjectId("66f..a1"), "user_id": ObjectId("66f..01"),
  "name": "Dr. Simran Kaur", "specialization": "General Physician",
  "languages": ["en", "hi", "pa"], "license_number": "PMC-12345",
  "council": "Punjab Medical Council",
  "verification_status": "verified", "verified_by": ObjectId("66f..ad"),
  "consultation_fee": 500, "is_active": true, "created_at": ISODate("2026-09-01T10:00:00Z") }

// doctor_availability (doctor picks slot length per block)
{ "doctor_id": ObjectId("66f..a1"), "day_of_week": 0,
  "start_time": "09:00", "end_time": "12:00", "timezone": "Asia/Kolkata",
  "slot_duration_min": 10, "is_active": true }

// appointments (times stored in UTC)
{ "patient_id": ObjectId("66f..p7"), "doctor_id": ObjectId("66f..a1"),
  "start_time": ISODate("2026-10-05T04:30:00Z"), "end_time": ISODate("2026-10-05T04:40:00Z"),
  "slot_duration_min": 10, "status": "scheduled",
  "reason": "fever since 2 days", "fee_snapshot": 500 }

// dicom_studies
{ "report_id": ObjectId("66f..r3"), "study_instance_uid": "1.2.840.113619.2...",
  "modality": "CR", "body_part": "CHEST", "study_date": "2026-09-28",
  "series": [{ "uid": "1.2.840...1", "instances": 2 }],
  "thumbnail_key": "patients/66f..p7/dicom/66f..r3/thumb.png", "anonymized": true }
```

7. Data flows for key features

Each flow is shown as a sequence diagram. Solid arrows are requests; dashed arrows are responses.

7.1 Registration and login

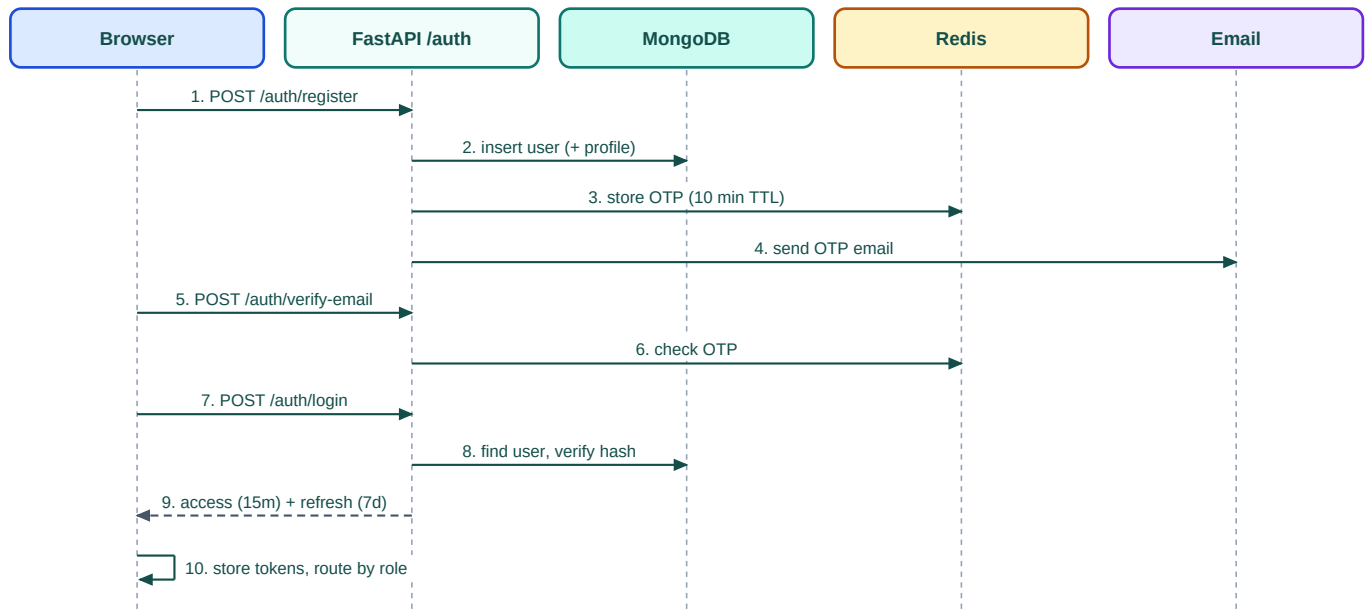


Figure 3 - Sign-up with OTP and JWT login

Access tokens are short-lived and kept in memory; the refresh token is sent as an `HttpOnly`, `Secure`, `SameSite` cookie and rotated on each refresh. Login and OTP endpoints are rate-limited by IP and by account.

7.2 Doctor onboarding and admin verification

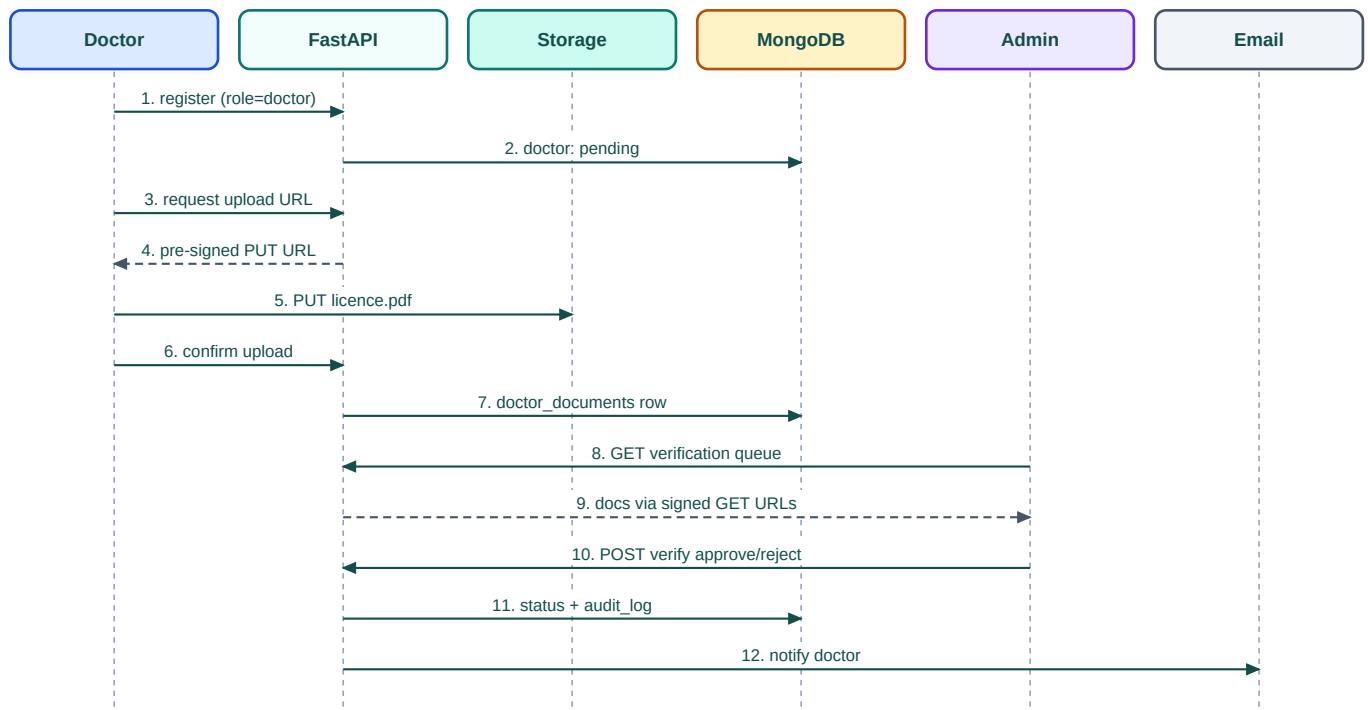


Figure 4 - Doctor verification lifecycle

Until `verification_status = verified` the doctor can edit their profile and documents but every availability, schedule and consultation endpoint returns **403 Doctor account not verified yet**. Rejected doctors see the admin's reason and can re-upload, which moves them back to *pending*.

7.3 Availability with custom slot duration

A verified doctor creates availability blocks and chooses the slot length for each block, so mornings can use 10-minute follow-ups and evenings 15-minute new consultations. The backend rejects overlapping blocks on the same day and any duration outside the allowed set. Slots are **computed on the fly** rather than stored, so changing a block never leaves stale rows.

```
# app/modules/availability/slots.py
def generate_slots(block, date_local, booked: set[datetime], now_utc, tz="Asia/Kolkata"):
    step = timedelta(minutes=block.slot_duration_min)
    start = localize(date_local, block.start_time, tz)
    end = localize(date_local, block.end_time, tz)
    slots, t = [], start
    while t + step <= end:
        s_utc = t.astimezone(UTC)
        if s_utc not in booked and s_utc > now_utc + timedelta(minutes=15):
            slots.append({"start": s_utc, "end": (t + step).astimezone(UTC)})
        t += step
    return slots

# 09:00-12:00 with 10 min -> 18 slots: 09:00-09:10, 09:10-09:20 ... 11:50-12:00
```

7.4 Booking an appointment

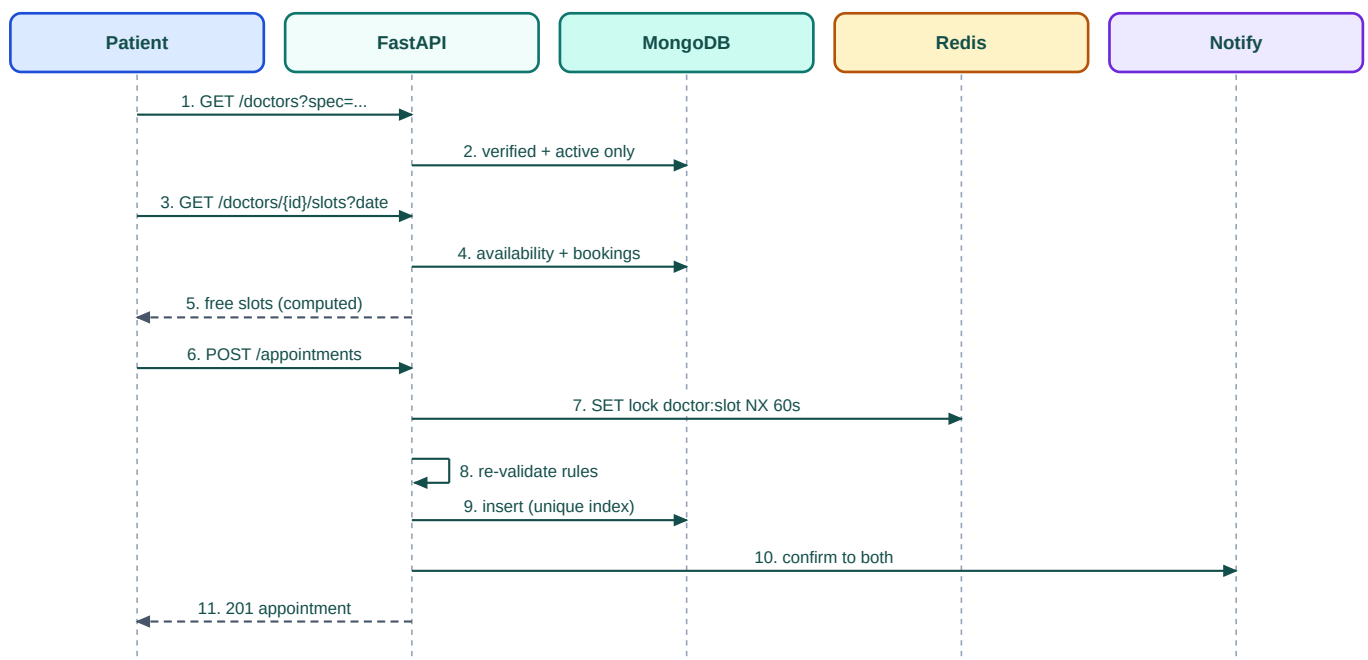


Figure 5 - Slot booking with double-booking protection

Step 8 re-checks that the doctor is verified and active, the slot is in the future, the slot aligns with a real availability block, and `end_time - start_time` equals that block's `slot_duration_min`. The Redis lock gives a friendly error quickly; the **unique partial index** on `{doctor_id, start_time}` is the final guarantee even if two requests race.

7.5 Video consultation

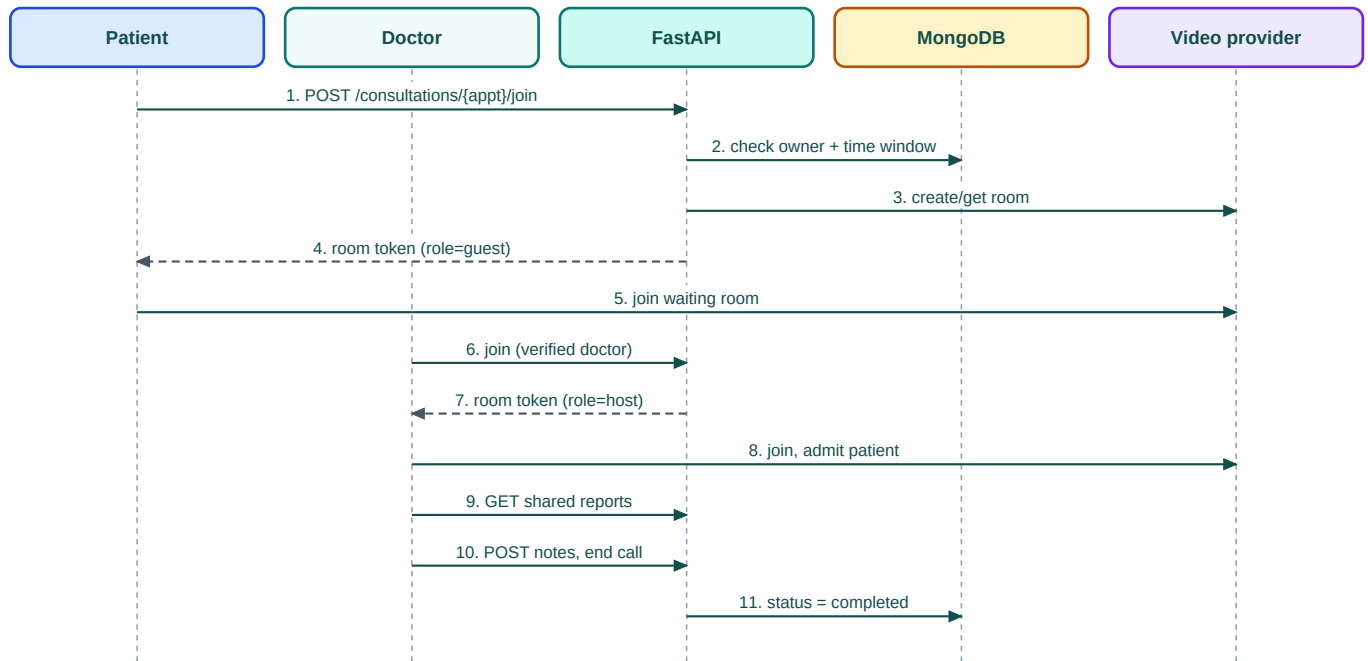


Figure 6 - Joining and running a video consultation

Tokens are only issued from 10 minutes before the start until the end of the slot plus a grace period, and only to the patient and doctor on that appointment. Media flows directly between browsers and the provider, never through Vercel.

7.6 Report and DICOM upload

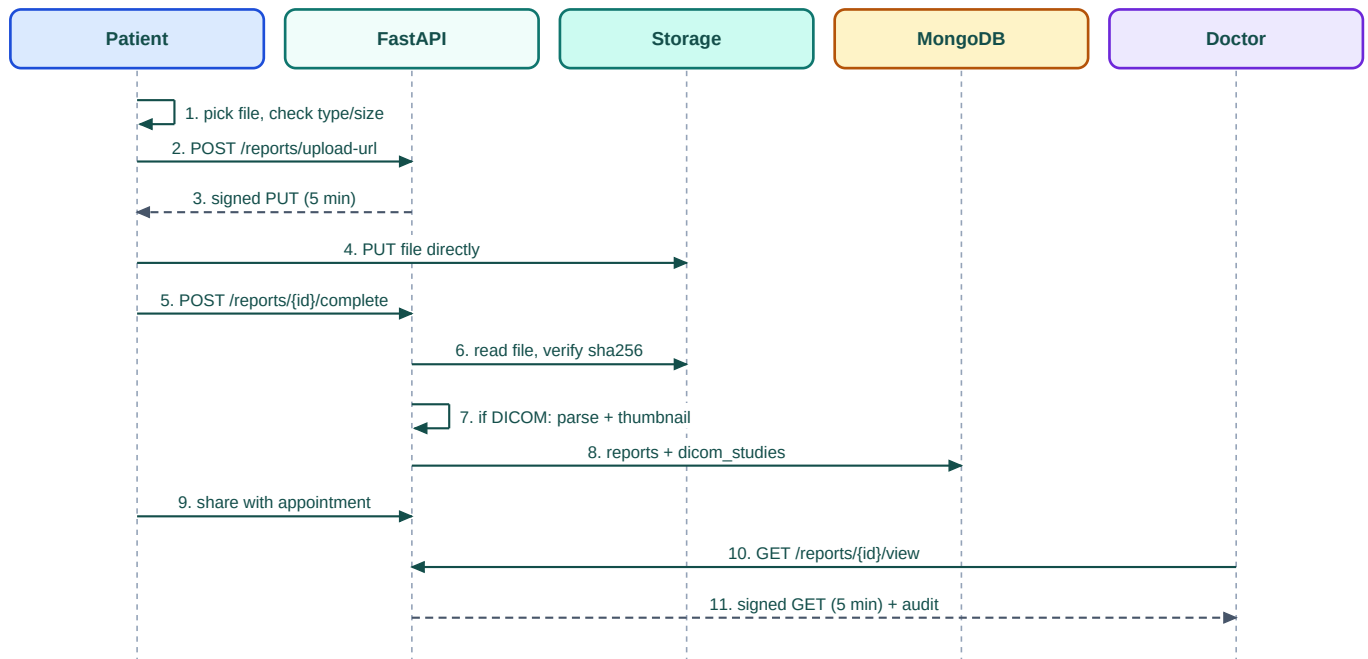


Figure 7 - Upload directly to storage, then register metadata

7.7 Chatbot query (RAG)

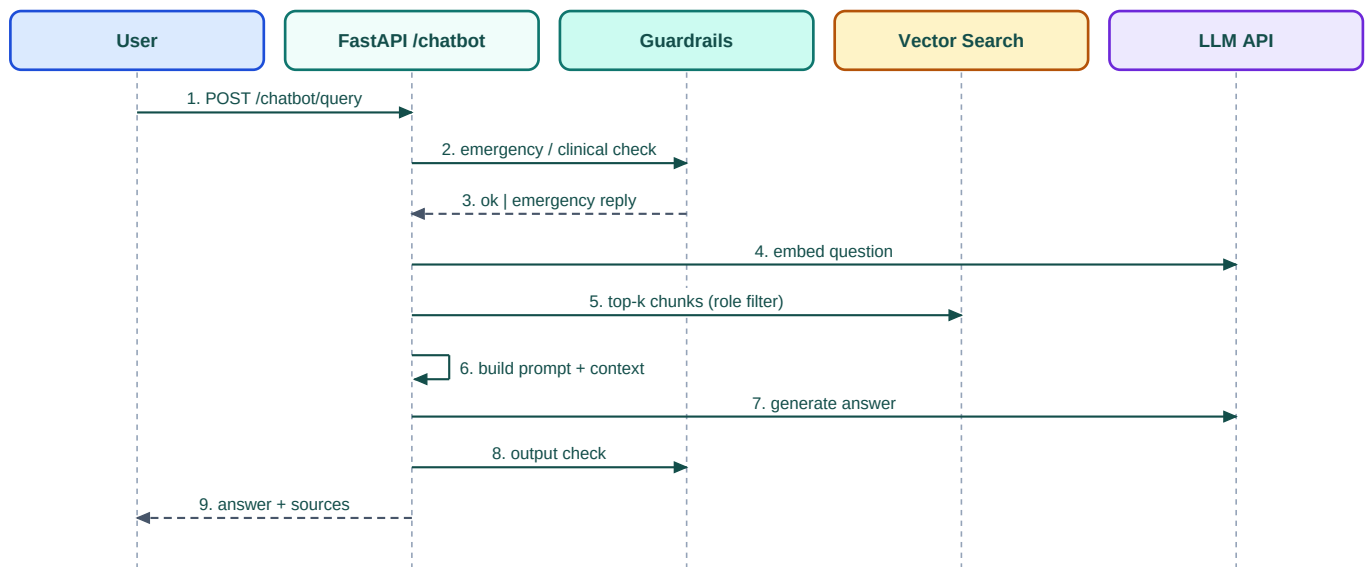


Figure 8 - Retrieval-augmented chatbot with guardrails

8. API reference (v1)

All routes are prefixed with `/api/v1`. FastAPI generates interactive docs at `/api/v1/docs`, which can be imported straight into Postman.

Method	Endpoint	Role	Description
POST	<code>/auth/register</code>	Public	Create patient or doctor account
POST	<code>/auth/verify-email</code>	Public	Confirm OTP
POST	<code>/auth/login</code>	Public	Returns access token, sets refresh cookie
POST	<code>/auth/refresh</code> <code>/auth/logout</code>	Any user	Rotate / revoke refresh token
POST	<code>/auth/forgot-password</code> <code>/auth/reset-password</code>	Public	Password reset via OTP
GET / PATCH	<code>/users/me</code>	Any user	Own account
GET / PUT	<code>/patients/me</code>	Patient	Own patient profile
GET	<code>/doctors</code>	Public	List verified doctors (filters, pagination)
GET	<code>/doctors/{id}</code>	Public	Public doctor profile
GET	<code>/doctors/{id}/slots?date=YYYY-MM-DD</code>	Public	Computed free slots for a date
GET / PUT	<code>/doctors/me</code>	Doctor	Own profile + verification status
POST	<code>/doctors/me/documents/upload-url</code>	Doctor	Pre-signed URL for licence upload
GET / POST	<code>/doctors/me/availability</code>	Verified doctor	List / create availability blocks
PATCH / DELETE	<code>/doctors/me/availability/{id}</code>	Verified doctor	Edit / disable a block
POST	<code>/doctors/me/leaves</code>	Verified doctor	Block a specific date
GET	<code>/doctors/me/appointments</code>	Verified doctor	Own schedule
POST	<code>/appointments</code>	Patient	Book a slot
GET	<code>/appointments/me</code>	Patient	Own appointments
PATCH	<code>/appointments/{id}/reschedule</code>	Patient	Move to another free slot
PATCH	<code>/appointments/{id}/cancel</code>	Patient, Doctor, Admin	Cancel with reason
POST	<code>/consultations/{appointment_id}/join</code>	Patient, Verified doctor	Get video room token
POST	<code>/consultations/{id}/end</code>	Verified doctor	Close consultation
POST / GET	<code>/consultations/{id}/notes</code>	Doctor (write), Patient (read)	SOAP notes
POST	<code>/reports/upload-url</code>	Patient	Pre-signed PUT URL
POST	<code>/reports/{id}/complete</code>	Patient	Register upload, parse DICOM
GET	<code>/reports/me</code>	Patient	Own reports
POST	<code>/reports/{id}/share</code>	Patient	Share with appointment's doctor
GET	<code>/reports/{id}/view</code>	Owner, shared doctor, admin	Short-lived signed GET URL (audited)
GET	<code>/dicom/{report_id}/metadata</code>	Owner, shared doctor	Study / series info for viewer
POST	<code>/chatbot/query</code>	Public (rate-limited)	Ask the assistant
GET	<code>/admin/doctors?status=pending</code>	Admin	Verification queue
POST	<code>/admin/doctors/{id}/verify</code>	Admin	action: approve reject (+reason) suspend
GET / PATCH	<code>/admin/users</code> <code>/admin/users/{id}/block</code>	Admin	User management
PATCH	<code>/admin/appointments/{id}</code>	Admin	Override / cancel
CRUD	<code>/admin/kb/articles</code>	Admin	Knowledge base (re-embeds on publish)
GET	<code>/admin/audit-logs</code> <code>/admin/analytics</code>	Admin	Logs and dashboard metrics
GET	<code>/health</code>	Public	Liveness + DB ping

8.1 Error format

```
HTTP 403
{ "error": { "code": "DOCTOR_NOT_VERIFIED",
             "message": "Doctor account not verified yet",
             "request_id": "c1f3b2..." } }
```

Common codes: VALIDATION_ERROR (422), INVALID_SLOT_DURATION (400), SLOT_TAKEN (409), DOCTOR_NOT_AVAILABLE (404), OUTSIDE_JOIN_WINDOW (403), RATE_LIMITED (429)

9. DICOM medical imaging

DICOM (*Digital Imaging and Communications in Medicine*) files contain both pixel data and a header of tags - including patient name, ID and birth date. That makes them medically useful but privacy-sensitive, so they get special handling.

Stage	What happens	Tools
Client check	Accept .dcm or .zip of a study; check size limit (e.g. 200 MB per study); read preamble 'DICM' marker	File API
Upload	Direct PUT to private bucket at patients/{pid}/dicom/{report_id}/	Pre-signed URL
Parse	Read tags: StudyInstanceUID, SeriesInstanceUID, Modality, BodyPartExamined, StudyDate, Rows/Columns	pydicom
Anonymise copy	Blank or replace identifying tags (PatientName, PatientID, BirthDate, InstitutionName) in the shared/viewer copy; keep original only if needed	pydicom
Thumbnail	Apply window/level, convert first frame to PNG for list views	numpy, Pillow
Store metadata	dicom_studies document linked 1:1 to reports	MongoDB
View	Doctor opens viewer; frontend fetches signed URLs and renders with window/level, zoom, pan, stack scroll, measurements	Cornerstone3D

Serverless caveat: large or multi-frame studies can exceed a function's memory or time limit. For the MVP, parse only headers with `stop_before_pixels=True` and build the thumbnail from one frame. If studies grow, move DICOM processing to a small background worker (e.g. Render or Railway) or add an Orthanc PACS server with DICOMweb, which the OHIF viewer can read directly.

10. LLM chatbot (RAG)

10.1 Content pipeline

- Admin writes or uploads an article in the admin console (FAQ, how-to, vetted health info).
- On publish, `kb/ingest.py` splits the Markdown into chunks of roughly 500 tokens with overlap, embeds each chunk and writes it to `kb_chunks`.
- Each chunk carries metadata such as `audience: [visitor, patient, doctor]` so retrieval can be filtered by the user's role.
- Editing an article deletes its old chunks and re-embeds; unpublishing removes them from search.

10.2 Query request

```
POST /api/v1/chatbot/query
{ "message": "How do I join my video consultation?",
  "session_id": "optional",
  "context": { "role": "patient", "appointment_id": "optional" } }

-> { "answer": "Open Appointments, then tap Join ...",
     "sources": [{ "title": "Joining a video call", "slug": "join-video-call" }],
     "flags": { "emergency": false, "clinical_redirect": false } }
```

10.3 Guardrails

Situation	Behaviour
Emergency keywords (chest pain, unconscious, suicidal thoughts, severe bleeding)	Skip the LLM; immediately show emergency message with 112 (India emergency number) and nearest-hospital advice
Diagnosis or prescription request	Give general, non-diagnostic information and suggest booking a consultation with a relevant specialist
Questions about a specific appointment	Answer from app data the user owns (time, join steps) - never from other users' data
No relevant context retrieved	Say it does not know and link to support rather than guessing
Personal health data in messages	Do not send PHI to the LLM beyond the question itself; store chat history only with consent and delete after a retention period

System prompt outline: *You are the AarogyaHub assistant. You help with using the app and share general, vetted health information from the provided context only. You never diagnose, prescribe or give dosages. If the user may be in danger, tell them to call 112 immediately.*

11. Security and compliance

Area	Measures
Transport	HTTPS everywhere (Vercel TLS); HSTS; strict CORS allow-list for the frontend domain
Authentication	bcrypt/argon2 password hashing; short-lived JWT access tokens; rotating HttpOnly refresh cookies; account logout after repeated failures
Authorization	Role checks + ownership checks on every query; verified-doctor dependency; admin actions audited
Files	Private bucket, no public URLs; pre-signed URLs expire in minutes; content-type and size validation; server-side encryption at rest
Database	Atlas network access list, least-privilege DB user, encryption at rest, automated backups
Input	Pydantic validation on all bodies; no raw query operators from user input (prevents NoSQL injection)
Rate limiting	Per-IP and per-user limits on login, OTP, chatbot and upload URL endpoints
Audit	Append-only audit_logs for verification decisions, report views, appointment overrides, user blocks
Secrets	Vercel environment variables only; never committed; separate keys per environment
Privacy (India)	Consent capture at signup, purpose-limited processing, data access and deletion requests, and breach handling per the DPDP Act, 2023
Telemedicine rules	Only registered practitioners consult; verify registration number against the medical council; record patient consent for teleconsultation; follow Telemedicine Practice Guidelines on prescribing

Regulations change and this is not legal advice - have a lawyer review the privacy policy, consent flows and prescribing features before launch.

12. Deployment on Vercel

Create **two Vercel projects from the same GitHub repo**: one with root directory frontend/ and one with root backend/. Each pull request gets its own preview URL.

```
// backend/vercel.json
{ "rewrites": [ { "source": "/(.*)", "destination": "/api/index.py" } ] }

# backend/api/index.py
from app.main import app          # Vercel detects the ASGI "app" object

// frontend/vercel.json (SPA routing)
{ "rewrites": [ { "source": "/(.*)", "destination": "/index.html" } ] }
```

12.1 Environment variables

Project	Variables
backend	MONGODB_URI, MONGODB_DB, JWT_SECRET, JWT_ACCESS_TTL_MIN, JWT_REFRESH_TTL_DAYS, CORS_ORIGINS, S3_ENDPOINT, S3_BUCKET, S3_ACCESS_KEY, S3_SECRET_KEY, UPSTASH_REDIS_REST_URL, UPSTASH_REDIS_REST_TOKEN, VIDEO_APP_ID, VIDEO_SECRET, LLM_API_KEY, EMBEDDING_MODEL, SENDGRID_API_KEY, SMS_API_KEY, SENTRY_DSN
frontend	VITE_API_URL, VITE_VIDEO_APP_ID, VITE_SENTRY_DSN

12.2 Serverless constraints and how the design handles them

Constraint	Impact	Design response
Request body size limit (about 4.5 MB)	Reports and DICOM cannot pass through the API	Browser uploads directly to object storage with pre-signed URLs
No persistent WebSocket server	Cannot self-host video signalling	Managed video provider handles signalling and TURN
Limited execution time and cold starts	Long jobs (large DICOM, bulk embedding) may time out	Keep functions short; move heavy jobs to a background worker when needed
Stateless instances	No in-memory sessions or locks	JWT for auth; Redis for locks and rate limits; reuse a global Mongo client between invocations

Alternative: if the backend outgrows serverless, deploy the same FastAPI app as a container on Render, Railway, Fly.io or AWS and keep the frontend on Vercel - no code changes beyond the entry point.

12.3 CI/CD pipeline

- Pull request: GitHub Actions runs ruff + mypy, pytest (against a test MongoDB), ESLint + Vitest, and Newman on the Postman collection.
- Vercel builds preview deployments for frontend and backend automatically.
- Merge to main: production deployment; scripts/create_indexes.py runs as a release step.
- Sentry captures errors from both apps with release tags.

13. Testing with Postman

MongoDB Atlas (free tier) or local MongoDB via Docker serves as the **testing database**, seeded with scripts/seed.py (one admin, verified and pending doctors, demo patients, sample KB articles).

AarogyaHub API (collection)	
00 Health	
01 Auth	register patient register doctor verify OTP login (saves {{token}}) refresh
02 Onboarding	get upload URL confirm document GET /doctors/me (status=pending)
03 Admin	login admin GET pending queue approve doctor reject doctor
04 Availability	create 09:00-12:00 / 10 min invalid duration 7 -> 400 unverified -> 403
05 Discovery	list doctors get slots for date (expect 18 slots)
06 Appointments	book slot book same slot -> 409 wrong duration -> 400 cancel
07 Consultation	join too early -> 403 join in window -> token
08 Reports+DICOM	upload URL complete share doctor view
09 Chatbot	FAQ question emergency phrase -> emergency flag
10 RBAC negatives	patient calls /admin -> 403 doctor reads other's report -> 404

Environment variable	Example
baseUrl	http://localhost:8000/api/v1 (local) https://api-aarogyahub.vercel.app/api/v1 (staging)
patientToken / doctorToken / adminToken	Set automatically by login test scripts
doctorId, appointmentId, reportId	Captured from responses with pm.environment.set(...)

```
// Tests tab of "Book slot"
pm.test("201 created", () => pm.response.to.have.status(201));
const body = pm.response.json();
pm.test("duration matches doctor setting", () => {
  const mins = (new Date(body.end_time) - new Date(body.start_time)) / 60000;
  pm.expect(mins).to.eql(10);
});
pm.environment.set("appointmentId", body.id);
```

14. Roadmap and milestones

Phase	Duration (indicative)	Deliverables
0. Setup	Week 1	Monorepo, CI, Vercel projects, Atlas cluster, seed script, Postman skeleton
1. Auth & roles	Weeks 2-3	Register/login/OTP, JWT refresh, RBAC dependencies, profile pages
2. Doctor onboarding	Weeks 3-4	Document upload, admin verification queue, notifications, audit logs
3. Scheduling	Weeks 5-6	Availability editor with slot duration, slot generation, booking, cancel/reschedule
4. Consultation	Weeks 7-8	Video provider integration, waiting room, notes
5. MedVault & DICOM	Weeks 8-9	Report upload, sharing, DICOM parsing, Cornerstone3D viewer
6. Chatbot	Week 10	KB editor, ingestion, RAG endpoint, guardrails, chat widget
7. Hardening	Weeks 11-12	Security review, load test booking, accessibility, legal review, launch
Later	-	Payments (Razorpay), e-prescriptions, MediCart pharmacy, MedGo mobile app, ABHA / ABDM integration, Orthanc PACS

Durations assume a small team of two to three developers and are a planning starting point, not a commitment.